

[YOUR COLLEGE NAME]
DEPARTMENT OF [YOUR DEPARTMENT]

Agentic AI Development Guide

A Complete, Step-by-Step Build Manual

AI-Powered Smart Farming Assistant

Submitted in partial fulfillment of the requirements for
9th Semester Mini Project

Submitted by:

[Student Name 1] – [Register Number]
[Student Name 2] – [Register Number]
[Student Name 3] – [Register Number]

Under the Guidance of:

[Guide Name]
[Designation]

[Month, Year]

Contents

1	Before You Start: The Concepts, in Plain Language	3
1.1	What is an "agent" in this context?	3
1.2	What is RAG?	3
1.3	What is an embedding, and why do we need a vector database?	3
1.4	What we are building, restated in one paragraph	3
2	Phase 1 — Accounts and Environment Setup	4
2.1	Step 1.1: Create a free Groq account	4
2.2	Step 1.2: Add the key to your backend	4
2.3	Step 1.3: Install the new Python packages	4
2.4	Step 1.4: New folder structure	4
3	Phase 2 — Building the Knowledge Base	6
3.1	Step 2.1: Download the KAU Package of Practices	6
3.2	Step 2.2: Enable pgvector on your Neon database	6
3.3	Step 2.3: Extract and chunk the PDF text	6
3.4	Step 2.4: Embed and store the chunks	7
4	Phase 3 — Wrapping the Existing ML Models as Tools	9
4.1	Step 3.1: Crop recommendation tool	9
4.2	Step 3.2: Yield prediction tool	9
5	Phase 4 — Structured Lookup Tools	11
5.1	Step 4.1: Calendar lookup tool	11
5.2	Step 4.2: Companion crop lookup tool	12
6	Phase 5 — The RAG Retrieval Tool	14
6.1	Step 5.1: Semantic search over the KAU knowledge base	14
7	Phase 6 — The Agent Orchestrator	15
7.1	Step 6.1: Tool schemas	15
7.2	Step 6.2: The system prompt	17
7.3	Step 6.3: The tool-calling loop	17
8	Phase 7 — The Backend Endpoint	20
8.1	Step 7.1: Schemas	20
8.2	Step 7.2: Router	20
8.3	Step 7.3: Wire into main.py	21
9	Phase 8 — Testing the Agent Locally	22
9.1	Step 8.1: Test each tool in isolation first	22
9.2	Step 8.2: Run the server and test the full loop	22
9.3	Step 8.3: What to check for	22

10 Phase 9 — Deploying the Agent (Render, Free)	23
10.1 Step 9.1: Add the new environment variable	23
10.2 Step 9.2: Update the build command	23
10.3 Step 9.3: Memory considerations	23
10.4 Step 9.4: Run the knowledge-base build script once, against production	23
11 Phase 10 — Mobile App Integration (Free)	24
11.1 Step 10.1: API client function	24
11.2 Step 10.2: The "Ask about any crop" input	24
11.3 Step 10.3: The reasoning trace panel	25
11.4 Step 10.4: Loading-state expectations	26
11.5 Step 10.5: Confirming the whole chain is genuinely free	26
12 Final Checklist	27

1 Before You Start: The Concepts, in Plain Language

This guide assumes zero prior experience with agents or RAG. Every phase after this one is code and commands – this section is the only place with pure explanation. Read it once; you won't need to re-read it while building.

1.1 What is an "agent" in this context?

An agent is a large language model (LLM) that, instead of just answering from what it already knows, is given a list of tools it can call – functions like “run the trained crop model” or “search our knowledge base” – and decides for itself, per question, which tools to call and in what order. The LLM never touches your database or your models directly; it just decides *which function to run*, your backend code actually runs it, and the result is fed back to the LLM so it can decide the next step or give a final answer. This back-and-forth is called the **tool-calling loop**, and Phase 6 builds it.

1.2 What is RAG?

RAG stands for Retrieval-Augmented Generation. Instead of asking an LLM to answer purely from its training data (which risks it inventing plausible-sounding but wrong agricultural advice – “hallucination”), you first **retrieve** real, relevant text from a trusted document (in our case, the KAU Package of Practices), then hand that retrieved text to the LLM and ask it to answer *using only that text*. The LLM's job shrinks from “know everything” to “read this and explain it clearly” – which is both more accurate and directly citable.

1.3 What is an embedding, and why do we need a vector database?

An embedding is a list of numbers (a vector) that represents the *meaning* of a piece of text. Similar meanings produce similar vectors. To “search by meaning” instead of by exact keyword, we convert every chunk of the KAU document into an embedding ahead of time and store it. Then, when a farmer asks a question, we embed *their question* the same way and ask the database: “which stored chunks have the closest vectors to this one?” That's semantic search, and pgvector is the Postgres extension that lets your existing Neon database do this natively – no separate vector database service needed.

1.4 What we are building, restated in one paragraph

An orchestrator (Groq's free LLM) that can call seven tools: two of your trained ML models, two existing API integrations (weather, market price), and three new tools (semantic search over KAU documents, a structured planting-calendar lookup, and a structured companion-crop lookup). The orchestrator decides which tools a given question needs, runs them, and produces one final answer in plain farmer-facing language – never exposing model internals in the main text, per the tone rules already defined in the proposal document.

2 Phase 1 — Accounts and Environment Setup

2.1 Step 1.1: Create a free Groq account

1. Go to <https://console.groq.com>
2. Sign up with email or Google – no credit card required
3. Navigate to **API Keys** in the left sidebar
4. Click **Create API Key**, name it (e.g. “smart-farming-agent”), copy the key immediately – it is only shown once

2.2 Step 1.2: Add the key to your backend

Open `.env` in your `backend_starter` project (the one from the earlier backend setup) and add:

```
GROQ_API_KEY=gsk_your_actual_key_here
```

Never commit this file – it should already be in `.gitignore` from the earlier backend setup.

2.3 Step 1.3: Install the new Python packages

From inside `backend_starter`, with your virtual environment activated:

```
pip install groq sentence-transformers PyMuPDF asyncpg
```

Add the same to `requirements.txt`:

```
groq==0.11.0
sentence-transformers==3.1.1
PyMuPDF==1.24.11
asyncpg==0.29.0
```

2.4 Step 1.4: New folder structure

Inside `backend_starter/app/`, create a new `agent/` package:

```
backend_starter/
  app/
    agent/
      __init__.py
      orchestrator.py
      tools/
        __init__.py
        crop_tool.py
        yield_tool.py
        calendar_tool.py
        companion_tool.py
        kau_search_tool.py
    rag/
      __init__.py
```

```
    ingest.py
    embed_utils.py
schemas/
    agent.py (new file, Phase 7)
routers/
    agent.py (new file, Phase 7)
```

Run `touch` on each `__init__.py` so Python treats these as packages:

```
mkdir -p app/agent/tools app/rag
touch app/agent/__init__.py app/agent/tools/__init__.py app/rag/__init__.py
```

3 Phase 2 — Building the Knowledge Base

3.1 Step 2.1: Download the KAU Package of Practices

Download the public PDF directly:

```
https://kau.in/sites/default/files/documents/pop2016.pdf
```

Save it as `backend_starter/data/kau_pop.pdf` (create the `data/` folder). This file is large (300+ pages covering all crops) – for the mini-project scope, it is fine to keep the whole thing; retrieval will naturally surface only the relevant pages per query.

3.2 Step 2.2: Enable pgvector on your Neon database

Open the Neon SQL editor (or connect via `psql` using your `DATABASE_URL`) and run:

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE IF NOT EXISTS kau_chunks (
  id SERIAL PRIMARY KEY,
  content TEXT NOT NULL,
  source_page INTEGER,
  embedding VECTOR(384)
);

CREATE INDEX IF NOT EXISTS kau_chunks_embedding_idx
  ON kau_chunks USING ivfflat (embedding vector_cosine_ops)
  WITH (lists = 100);
```

`VECTOR(384)` matches the output size of the `all-MiniLM-L6-v2` embedding model chosen for this project. This table is deliberately *not* added to `prisma/schema.prisma` – Prisma’s Python client does not reliably support the `vector` type or the cosine-distance operator (`<=>`) needed for similarity search, so this table is created and queried with raw SQL via `asyncpg` instead. Your other 14 tables stay exactly as Prisma-managed as before.

3.3 Step 2.3: Extract and chunk the PDF text

Create `app/rag/ingest.py`:

```
"""
One-time script: extracts text from the KAU PDF, splits it into overlapping
chunks, and returns them ready for embedding. Run manually, not on every
server startup -- see Phase 2 Step 2.5 for how to run this.
"""
import fitz # PyMuPDF

CHUNK_SIZE = 800 # characters per chunk
CHUNK_OVERLAP = 150 # overlap so a fact split across a chunk boundary isn't lost

def extract_and_chunk(pdf_path: str):
```

```
doc = fitz.open(pdf_path)
chunks = []

for page_num, page in enumerate(doc, start=1):
    text = page.get_text().strip()
    if not text:
        continue

    start = 0
    while start < len(text):
        end = start + CHUNK_SIZE
        chunk_text = text[start:end].strip()
        if len(chunk_text) > 50: # skip near-empty fragments
            chunks.append({"content": chunk_text, "source_page": page_num})
            start += CHUNK_SIZE - CHUNK_OVERLAP

doc.close()
print(f"Extracted {len(chunks)} chunks from {pdf_path}")
return chunks
```

3.4 Step 2.4: Embed and store the chunks

Create `app/rag/embedutils.py`:

```
"""
Wraps the sentence-transformers embedding model as a single reusable object,
loaded once, not per-call.
"""
from sentence_transformers import SentenceTransformer

_model = None

def get_embedding_model():
    global _model
    if _model is None:
        print("Loading embedding model (all-MiniLM-L6-v2)...")
        _model = SentenceTransformer("all-MiniLM-L6-v2")
    return _model

def embed_text(text: str):
    model = get_embedding_model()
    return model.encode(text).tolist()
```

Create a one-time run script `scripts/build_knowledge_base.py` at the project root:

```
"""
Run this ONCE to populate kau_chunks. Re-run only if the source PDF changes.
Usage: python scripts/build_knowledge_base.py
"""
import asyncio
```



```
import asyncpg
import os
from dotenv import load_dotenv

from app.rag.ingest import extract_and_chunk
from app.rag.embed_utils import embed_text

load_dotenv()

async def main():
    chunks = extract_and_chunk("data/kau_pop.pdf")

    conn = await asyncpg.connect(os.environ["DATABASE_URL"])

    # Clear old data if re-running
    await conn.execute("TRUNCATE TABLE kau_chunks RESTART IDENTITY;")

    for i, chunk in enumerate(chunks):
        vector = embed_text(chunk["content"])
        # asyncpg needs the vector as a string in pgvector's literal format
        vector_str = "[" + ",".join(str(x) for x in vector) + "]"
        await conn.execute(
            "INSERT INTO kau_chunks (content, source_page, embedding) VALUES ($1, $2, "
            "$3::vector)",
            chunk["content"], chunk["source_page"], vector_str,
        )
        if i % 50 == 0:
            print(f"Inserted {i}/{len(chunks)} chunks...")

    await conn.close()
    print("Knowledge base build complete.")

if __name__ == "__main__":
    asyncio.run(main())
```

Run it once:

```
python scripts/build_knowledge_base.py
```

This will take a few minutes depending on PDF size – it's a one-time cost, not something that runs on every server start.

4 Phase 3 — Wrapping the Existing ML Models as Tools

4.1 Step 3.1: Crop recommendation tool

This wraps the `CropRecommendationModel` class you already built and tested – no changes to that class itself. Create `app/agent/tools/crop_tool.py`:

```
"""
Thin wrapper: the agent calls this function by name; internally it just
calls your already-working CropRecommendationModel.
"""

# Model 1 only knows these 22 crops. The agent's system prompt (Phase 6)
# is told this explicitly so it never calls this tool for an unsupported crop.
SUPPORTED_CROPS = [
    "rice", "maize", "chickpea", "kidneybeans", "pigeonpeas", "mothbeans",
    "mungbean", "blackgram", "lentil", "pomegranate", "banana", "mango",
    "grapes", "watermelon", "muskmelon", "apple", "orange", "papaya",
    "coconut", "cotton", "jute", "coffee",
]

def run_crop_recommendation(crop_model, N, P, K, temperature, humidity, ph,
    rainfall):
    """crop_model is the already-loaded CropRecommendationModel instance
    from app.state (see main.py) -- passed in, not re-loaded here."""
    return crop_model.predict(
        N=N, P=P, K=K, temperature=temperature,
        humidity=humidity, ph=ph, rainfall=rainfall,
    )
```

4.2 Step 3.2: Yield prediction tool

This wraps the two-stage pipeline from your yield-prediction README (Model 2). Create `app/agent/tools/yield`:

```
"""
Wraps the yield prediction pipeline. Assumes Model 2 (yield_model.pkl and its
encoder) has already been trained per the earlier pipeline documentation and
placed in ml_models/yield_prediction/, following the same pattern as Model 1.
"""

import joblib
import os
import pandas as pd

_yield_model = None
_yield_encoder = None

def load_yield_model(model_dir="ml_models/yield_prediction"):
    global _yield_model, _yield_encoder
    if _yield_model is None:
        _yield_model = joblib.load(os.path.join(model_dir, "crop_yield_model.pkl"))
```

```
    _yield_encoder = joblib.load(os.path.join(model_dir, "encoder.pkl"))
    return _yield_model, _yield_encoder

def run_yield_prediction(state, district, crop, temperature, humidity, rainfall,
    farm_area):
    model, encoder = load_yield_model()

    input_df = pd.DataFrame([{"State": state, "District": district, "Crop": crop,
        "temperature": temperature, "humidity": humidity, "rainfall": rainfall,
    }])
    encoded_input = encoder.transform(input_df)
    yield_per_hectare = float(model.predict(encoded_input)[0])
    total_estimated_harvest = yield_per_hectare * farm_area

    return {
        "yield_per_hectare": round(yield_per_hectare, 2),
        "total_estimated_harvest": round(total_estimated_harvest, 2),
    }
```

Adjust the exact file/column names above once Model 2 is actually trained – the shape shown here matches the pipeline design from your earlier README.

5 Phase 4 — Structured Lookup Tools

5.1 Step 4.1: Calendar lookup tool

Create `app/agent/tools/calendar_tool.py`:

```
"""
Structured lookup, not semantic search -- calendar facts should be exact.
Expand this table over time as you cover more crops.
"""

CROP_CALENDAR = {
    "rice_virippu": {
        "sowing_months": ["Edavam", "Midhunam"],
        "sowing_gregorian": "mid-May to mid-July",
        "harvest_months": ["Chingam", "Kanni"],
        "harvest_gregorian": "mid-August to mid-October",
    },
    "rice_mundakan": {
        "sowing_months": ["Chingam", "Kanni"],
        "sowing_gregorian": "mid-August to mid-October",
        "harvest_months": ["Vrischikam", "Dhanu"],
        "harvest_gregorian": "mid-November to mid-January",
    },
    "rice_puncha": {
        "sowing_months": ["Dhanu", "Makaram"],
        "sowing_gregorian": "mid-December to mid-February",
        "harvest_months": ["Meenam", "Medam"],
        "harvest_gregorian": "mid-March to mid-May",
    },
    "coconut": {
        "sowing_months": ["Edavam", "Midhunam"],
        "sowing_gregorian": "mid-May to mid-July",
        "harvest_months": [],
        "harvest_gregorian": "year-round once bearing (every 45-60 days)",
    },
    "pepper": {
        "sowing_months": ["Edavam", "Midhunam"],
        "sowing_gregorian": "mid-May to mid-July",
        "harvest_months": ["Makaram", "Kumbham"],
        "harvest_gregorian": "mid-January to mid-March",
    },
    "banana": {
        "sowing_months": ["Chingam", "Kanni"],
        "sowing_gregorian": "mid-August to mid-October (for Onam-season harvest)",
        "harvest_months": [],
        "harvest_gregorian": "year-round, 10-12 months after planting",
    },
    # Add more crops here as the knowledge base grows.
}

def lookup_calendar(crop_name: str):
```

```

key = crop_name.lower().strip().replace(" ", "_")
if key in CROP_CALENDAR:
    return CROP_CALENDAR[key]

# Fuzzy fallback: match "rice" to the first rice entry found
for k, v in CROP_CALENDAR.items():
    if key in k:
        return v

return {"error": f"No calendar entry for '{crop_name}' yet -- fall back to
    kau_knowledge_search."}

```

5.2 Step 4.2: Companion crop lookup tool

Create `app/agent/tools/companion_tool.py`:

```

"""
Curated folk-wisdom companion planting pairs -- a second, independent
knowledge source alongside RAG, not a replacement for it.
"""

COMPANION_RULES = {
    "bitter_gourd": [
        {"companion": "yardlong beans / cowpea", "reasoning": "Shared trellis, beans
            fix nitrogen the gourd needs, staggered harvest timing."}
    ],
    "ridge_gourd": [
        {"companion": "amaranthus", "reasoning": "Fast-growing leafy crop fills
            ground space while the vine establishes overhead."}
    ],
    "ash_gourd": [
        {"companion": "cowpea", "reasoning": "Nitrogen-fixing companion; both
            tolerate similar watering schedules."}
    ],
    "okra": [
        {"companion": "beans", "reasoning": "Complementary root depth -- beans
            shallow, okra deep -- reduces competition."}
    ],
    "coconut": [
        {"companion": "pepper", "reasoning": "Traditional multi-tier Kerala
            homestead system."},
        {"companion": "banana", "reasoning": "Traditional multi-tier Kerala
            homestead system."},
        {"companion": "cocoa", "reasoning": "Traditional multi-tier Kerala homestead
            system."},
    ],
    "rice": [
        {"companion": "fish (integrated rice-fish)", "reasoning": "Traditional
            Kuttanad practice -- fish control pests, add nutrients to paddy water."}
    ],
    # Add more pairs as you research -- aim for 15-20 total.
}

```

```
def lookup_companions(crop_name: str):  
    key = crop_name.lower().strip().replace(" ", "_").replace("gourd", "gourd")  
    return COMPANION_RULES.get(key, [])
```

6 Phase 5 — The RAG Retrieval Tool

6.1 Step 5.1: Semantic search over the KAU knowledge base

Create `app/agent/tools/kau-search-tool.py`:

```
"""
Embeds the incoming query, then asks Postgres (via pgvector) for the chunks
whose stored embeddings are closest by cosine distance. This is the tool
that lets the agent answer questions about crops outside Model 1's training
set -- pepper, cardamom, rubber, tapioca, etc.
"""
import asyncpg
import os

from app.rag.embed_utils import embed_text

async def search_kau_knowledge(query: str, top_k: int = 3):
    query_vector = embed_text(query)
    vector_str = "[" + ",".join(str(x) for x in query_vector) + "]"

    conn = await asyncpg.connect(os.environ["DATABASE_URL"])
    try:
        rows = await conn.fetch(
            """
            SELECT content, source_page,
                   1 - (embedding <=> $1::vector) AS similarity
            FROM kau_chunks
            ORDER BY embedding <=> $1::vector
            LIMIT $2
            """,
            vector_str, top_k,
        )
    finally:
        await conn.close()

    return [
        {
            "content": row["content"],
            "source_page": row["source_page"],
            "similarity": round(float(row["similarity"]), 3),
        }
        for row in rows
    ]
```

The `<=>` operator is `pgvector`'s cosine-distance operator – lower distance means more similar meaning. This is exactly the query that Prisma's Python client cannot reliably express, which is why this tool uses `asyncpg` directly instead.

7 Phase 6 — The Agent Orchestrator

This is the core of the whole system: the loop that lets the LLM decide which tools to call.

7.1 Step 6.1: Tool schemas

Create `app/agent/orchestrator.py` and start with the tool definitions – these are the “menu” of what the LLM is allowed to call, in OpenAI-compatible function-calling format (Groq uses the same format):

```
TOOLS = [
    {
        "type": "function",
        "function": {
            "name": "crop_recommendation_model",
            "description": "Trained Random Forest classifier. ONLY covers these 22 crops -- never call for any other crop: rice, maize, chickpea, kidneybeans, pigeonpeas, mothbeans, mungbean, blackgram, lentil, pomegranate, banana, mango, grapes, watermelon, muskmelon, apple, orange, papaya, coconut, cotton, jute, coffee.",
            "parameters": {
                "type": "object",
                "properties": {
                    "N": {"type": "number"}, "P": {"type": "number"}, "K": {"type": "number"},
                    "temperature": {"type": "number"}, "humidity": {"type": "number"},
                    "ph": {"type": "number"}, "rainfall": {"type": "number"},
                },
                "required": ["N", "P", "K", "temperature", "humidity", "ph", "rainfall"],
            },
        },
    },
    {
        "type": "function",
        "function": {
            "name": "yield_prediction_model",
            "description": "Predicts expected yield per hectare and total harvest for a given crop, location, and farm area.",
            "parameters": {
                "type": "object",
                "properties": {
                    "state": {"type": "string"}, "district": {"type": "string"},
                    "crop": {"type": "string"}, "temperature": {"type": "number"},
                    "humidity": {"type": "number"}, "rainfall": {"type": "number"},
                    "farm_area": {"type": "number", "description": "Farm area in hectares"},
                },
                "required": ["state", "district", "crop", "temperature", "humidity", "rainfall", "farm_area"],
            },
        },
    },
]
```



```

    },
    {
      "type": "function",
      "function": {
        "name": "crop_calendar_lookup",
        "description": "Returns the traditional Malayalam-calendar planting and
          harvesting window for a crop. Always call this before discussing
          timing -- lead with tradition, not weather.",
        "parameters": {
          "type": "object",
          "properties": {"crop_name": {"type": "string"}},
          "required": ["crop_name"],
        },
      },
    },
    {
      "type": "function",
      "function": {
        "name": "companion_rules_lookup",
        "description": "Looks up traditional/folk companion-planting pairs for a
          crop. Call this FIRST when suggesting intercrops, before
          kau_knowledge_search.",
        "parameters": {
          "type": "object",
          "properties": {"crop_name": {"type": "string"}},
          "required": ["crop_name"],
        },
      },
    },
    {
      "type": "function",
      "function": {
        "name": "kau_knowledge_search",
        "description": "Semantic search over Kerala Agricultural University's
          Package of Practices. Use for any crop NOT covered by
          crop_recommendation_model, and to add supporting agronomic detail
          after companion_rules_lookup.",
        "parameters": {
          "type": "object",
          "properties": {
            "query": {"type": "string"},
            "top_k": {"type": "integer", "default": 3},
          },
          "required": ["query"],
        },
      },
    },
  ],
]

```

Note: `weather_lookup` and `market_price_lookup` already exist as internal functions from earlier backend phases – add them to this same `TOOLS` list following the identical pattern once you reach integration; omitted here only to keep this listing focused on the newly built pieces.

7.2 Step 6.2: The system prompt

This is the consolidated version combining every rule from the proposal document (tool-usage rules) and the tone document (voice rules):

```
SYSTEM_PROMPT = """
You are Krishi Sakhi, an agricultural reasoning assistant supporting Kerala
farmers. You have access to tools -- use them, never answer from memory alone
for anything crop-specific.

Rules:
1. For crops covered by crop_recommendation_model's known class list, ALWAYS
   call it first and treat its output as your primary, ML-verified answer.
2. For crops NOT covered by the trained model, do NOT guess -- call
   kau_knowledge_search before answering. Never fabricate agronomic advice.
3. Always call crop_calendar_lookup before discussing planting timing. Lead
   with the traditional Malayalam month window; treat weather only as a
   secondary caution, never as the primary timing driver.
4. Always call companion_rules_lookup before suggesting intercrops. Use the
   folk-wisdom pairing as the headline; use kau_knowledge_search only to add
   supporting detail underneath.
5. Call market_price_lookup when the question involves selling decisions.
6. Clearly label each part of your answer as "ML-Verified" or "AI-Reasoned
   from agricultural literature" internally in your structured output -- but
   NEVER say "As an AI," "based on my analysis," or cite confidence scores
   in the natural-language answer text itself.
7. Speak with the plain confidence of an experienced local farmer or
   agricultural extension officer. Reference Malayalam month names alongside
   the Gregorian range. Use concrete, sensory language over statistics --
   "harvest when the pods snap easily" rather than "harvest at maturity."
8. Cite the specific KAU source page whenever kau_knowledge_search is used.
9. If no tool can answer confidently, say so plainly. Do not fabricate.
10. Keep the final answer concise and actionable.
"""
```

7.3 Step 6.3: The tool-calling loop

This is the actual orchestration logic – the part that makes this an "agent" rather than a single API call:

```
import json
from groq import Groq

from app.agent.tools.crop_tool import run_crop_recommendation
from app.agent.tools.yield_tool import run_yield_prediction
from app.agent.tools.calendar_tool import lookup_calendar
from app.agent.tools.companion_tool import lookup_companions
from app.agent.tools.kau_search_tool import search_kau_knowledge

client = Groq() # reads GROQ_API_KEY from the environment automatically

MODEL_NAME = "llama-3.3-70b-versatile"
```

```

async def execute_tool_call(tool_name: str, arguments: dict, crop_model):
    """Routes a tool call from the LLM to the actual Python function."""
    if tool_name == "crop_recommendation_model":
        return run_crop_recommendation(crop_model, **arguments)
    elif tool_name == "yield_prediction_model":
        return run_yield_prediction(**arguments)
    elif tool_name == "crop_calendar_lookup":
        return lookup_calendar(arguments["crop_name"])
    elif tool_name == "companion_rules_lookup":
        return lookup_companions(arguments["crop_name"])
    elif tool_name == "kau_knowledge_search":
        return await search_kau_knowledge(**arguments)
    else:
        return {"error": f"Unknown tool: {tool_name}"}

async def run_agent(user_message: str, crop_model, max_turns: int = 6):
    """
    The core agent loop. Returns the final answer plus a reasoning trace
    (which tools were called, in what order) for the UI to display.
    """
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": user_message},
    ]
    reasoning_trace = []

    for _ in range(max_turns):
        response = client.chat.completions.create(
            model=MODEL_NAME,
            messages=messages,
            tools=TOOLS,
            tool_choice="auto",
        )
        message = response.choices[0].message

        if not message.tool_calls:
            # No more tools needed -- this is the final answer.
            return {
                "answer": message.content,
                "reasoning_trace": reasoning_trace,
            }

        # The LLM wants to call one or more tools -- run them.
        messages.append(message)
        for tool_call in message.tool_calls:
            tool_name = tool_call.function.name
            arguments = json.loads(tool_call.function.arguments)

            result = await execute_tool_call(tool_name, arguments, crop_model)
            reasoning_trace.append({"tool": tool_name, "arguments": arguments, "

```

```
        result_summary": str(result)[:200]})

    messages.append({
        "role": "tool",
        "tool_call_id": tool_call.id,
        "content": json.dumps(result),
    })

    return {
        "answer": "I wasn't able to reach a confident answer within the available\n        steps -- please try rephrasing your question.",
        "reasoning_trace": reasoning_trace,
    }
```

How this loop works, in plain terms: every iteration, the LLM either says "I need to call these tools" (in which case your code runs them and feeds the results back in) or says "here is my final answer" (in which case the loop stops). `max_turns` is a safety cap so a confused agent can't loop forever and burn your free-tier quota.

8 Phase 7 — The Backend Endpoint

8.1 Step 7.1: Schemas

Create `app/schemas/agent.py`:

```
from pydantic import BaseModel
from typing import List, Dict, Any

class AgentRequest(BaseModel):
    message: str
    farm_id: str | None = None

class ReasoningStep(BaseModel):
    tool: str
    arguments: Dict[str, Any]
    result_summary: str

class AgentResponse(BaseModel):
    answer: str
    reasoning_trace: List[ReasoningStep]
```

8.2 Step 7.2: Router

Create `app/routers/agent.py`:

```
from fastapi import APIRouter, Request, HTTPException

from app.schemas.agent import AgentRequest, AgentResponse
from app.agent.orchestrator import run_agent

router = APIRouter()

@router.post("/agent/crop-advisory", response_model=AgentResponse)
async def crop_advisory(payload: AgentRequest, request: Request):
    """
    The agentic endpoint. Existing /crop-recommendation endpoint (Model 1
    only) stays untouched as a fast, simple fallback path.
    """
    crop_model = request.app.state.crop_model
    try:
        result = await run_agent(payload.message, crop_model)
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Agent failed: {e}")

    return result
```

8.3 Step 7.3: Wire into main.py

Add one import and one line to the existing `main.py`:

```
from app.routers import agent as agent_router
# ... inside the existing app setup, alongside the crop router ...
app.include_router(agent_router.router, prefix="/api/v1", tags=["Agent"])
```

9 Phase 8 — Testing the Agent Locally

9.1 Step 8.1: Test each tool in isolation first

Before testing the full agent loop, verify each tool function works on its own – this makes debugging far easier than only testing through the LLM:

```
python3 -c "  
from app.agent.tools.calendar_tool import lookup_calendar  
print(lookup_calendar('coconut'))  
"
```

Repeat similarly for `lookup_companions`, and for `search_kau_knowledge` (this one needs `asyncio.run(...)` since it's async).

9.2 Step 8.2: Run the server and test the full loop

```
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Scenario A – crop in the training set:

```
curl -X POST http://localhost:8000/api/v1/agent/crop-advisory \  
-H "Content-Type: application/json" \  
-d '{"message": "My soil has N=40 P=30 K=35 pH=5.8, 1 acre in Kottayam. What  
should I plant, and what grows well alongside it?"}'
```

Scenario B – crop outside the training set:

```
curl -X POST http://localhost:8000/api/v1/agent/crop-advisory \  
-H "Content-Type: application/json" \  
-d '{"message": "Is my land in Wayanad suitable for growing cardamom, and when  
should I plant it?"}'
```

9.3 Step 8.3: What to check for

- **Scenario A** should show `crop_recommendation_model` in the reasoning trace, not `kau_knowledge_search` as the primary source.
- **Scenario B** should show `kau_knowledge_search` being called, and `crop_recommendation_model` should *not* appear (cardamom isn't in its class list).
- Both should show `crop_calendar_lookup` being called before any timing advice appears.
- The `answer` text should read like the "sounds like an expert" example, not the "sounds like an AI" one, from the tone guide.
- If a tool call errors, check that error appears clearly in your terminal logs – silent failures are the hardest to debug in agent systems.

10 Phase 9 — Deploying the Agent (Render, Free)

10.1 Step 9.1: Add the new environment variable

In the Render dashboard, under your existing web service's Environment tab, add:

```
GROQ_API_KEY = gsk_your_actual_key_here
```

10.2 Step 9.2: Update the build command

The embedding model should be downloaded once during build, not on every cold start:

```
pip install -r requirements.txt && prisma generate && prisma db push && python  
download_models.py && python -c "from sentence_transformers import  
SentenceTransformer; SentenceTransformer('all-MiniLM-L6-v2')"
```

That last piece pre-downloads and caches the embedding model into the build image, so the first real request doesn't pay that download cost.

10.3 Step 9.3: Memory considerations

Render's free tier gives 512MB RAM. Rough budget: FastAPI + scikit-learn model $\approx$ 150MB, sentence-transformers model $\approx$ 90MB in memory once loaded, plus normal Python/Uvicorn overhead. This should fit, but if you see out-of-memory crashes after adding the embedding model, the first thing to check is whether `get_embedding_model()` (Phase 2, Step 2.4) is genuinely only loading once – a bug that reloads it per-request would exhaust free-tier memory quickly.

10.4 Step 9.4: Run the knowledge-base build script once, against production

The `kau_chunks` table needs to be populated in your actual Neon database, not just your local one (unless you're using the same Neon database for both, which is common and fine for a mini project). Run `scripts/build_knowledge_base.py` once locally, pointed at the same `DATABASE_URL` your Render deployment uses – there's no need to run this inside Render itself.

11 Phase 10 — Mobile App Integration (Free)

There is no separate "deployment" step for the agent on the mobile side. The agent lives entirely on your already-free Render backend; the mobile app simply calls it over HTTPS, exactly like every other endpoint already built. This phase is about the React Native code to call it and display the result well.

11.1 Step 10.1: API client function

In your Expo project, wherever existing API calls live (e.g. `api/client.ts`):

```
export async function askCropAdvisor(message: string, farmId?: string) {
  const response = await fetch(`${API_BASE_URL}/api/v1/agent/crop-advisory`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ message, farm_id: farmId }),
  });

  if (!response.ok) {
    throw new Error("Advisor request failed");
  }
  return response.json(); // { answer, reasoning_trace }
}
```

11.2 Step 10.2: The "Ask about any crop" input

A minimal component using your existing NativeWind styling conventions:

```
import { useState } from "react";
import { View, TextInput, Pressable, Text, ActivityIndicator } from "react-native";
import { askCropAdvisor } from "../api/client";

export function CropAdvisorInput() {
  const [query, setQuery] = useState("");
  const [loading, setLoading] = useState(false);
  const [result, setResult] = useState<any>(null);

  async function handleAsk() {
    setLoading(true);
    try {
      const res = await askCropAdvisor(query);
      setResult(res);
    } catch (e) {
      console.error(e);
    } finally {
      setLoading(false);
    }
  }

  return (
    <View className="p-4">
```

```

<TextInput
  className="border border-gray-300 rounded-2xl px-4 py-3 text-sm"
  placeholder="Ask about any crop..."
  value={query}
  onChangeText={setQuery}
/>
<Pressable onPress={handleAsk} className="bg-emerald-700 rounded-2xl py-3 mt-3">
  <Text className="text-white text-center font-bold">Ask</Text>
</Pressable>

{loading && <ActivityIndicator className="mt-4" />}

{result && (
  <View className="mt-4">
    <Text className="text-sm leading-relaxed">{result.answer}</Text>
    <ReasoningTracePanel steps={result.reasoning_trace} />
  </View>
)}
</View>
);
}

```

11.3 Step 10.3: The reasoning trace panel

This is the "how I decided" collapsible element from the UI-changes plan:

```

import { useState } from "react";
import { View, Text, Pressable } from "react-native";

export function ReasoningTracePanel({ steps }: { steps: any[] }) {
  const [open, setOpen] = useState(false);

  return (
    <View className="mt-3 border border-gray-200 rounded-xl p-3">
      <Pressable onPress={() => setOpen(!open)}>
        <Text className="text-xs font-bold text-emerald-700">
          {open ? "Hide" : "Show"} how I decided ({steps.length} steps)
        </Text>
      </Pressable>
      {open && steps.map((step, i) => (
        <Text key={i} className="text-[11px] text-gray-500 mt-1">
          {i + 1}. Checked {step.tool.replace(/_/g, " ")}
        </Text>
      ))}
    </View>
  );
}

```

11.4 Step 10.4: Loading-state expectations

An agent response involves multiple sequential tool calls plus at least two LLM round-trips (one to decide which tools to call, one to produce the final answer) – realistically 3–8 seconds, noticeably slower than the single-model `/crop-recommendation` endpoint. Keep the loading indicator visible for that whole window, and consider a short rotating status line (“Checking your soil data..”, “Looking up traditional planting calendar..”) rather than a bare spinner, so the wait feels purposeful rather than broken.

11.5 Step 10.5: Confirming the whole chain is genuinely free

Component	Cost
Groq API (LLM)	Free tier, no card, no expiry
Render (backend hosting)	Free tier
Neon (Postgres + pgvector)	Free tier
sentence-transformers (embeddings)	Free, open source, runs locally
KAU Package of Practices	Free public document
Expo (mobile app)	Free tier

Every piece of this system runs at \$0. The only real constraints are the free-tier *rate limits* (Groq: 30 requests/minute), which are far above what a classroom demo or review panel will generate.

12 Final Checklist

Groq account created, API key in `.env`

New packages installed and added to `requirements.txt`

`app/agent/` and `app/rag/` folder structure created

KAU PDF downloaded, `pgvector` enabled on Neon, `kau_chunks` table created

`build_knowledge_base.py` run successfully, chunks populated

Model 1 and Model 2 wrapped as tool functions

Calendar and companion lookup tables populated with at least 6–10 crops each

`kau_search_tool.py` tested independently, returns relevant chunks

`orchestrator.py` built: tool schemas, system prompt, tool-calling loop

New `/api/v1/agent/crop-advisory` endpoint wired into `main.py`

Both test scenarios (crop-in-list, crop-not-in-list) verified locally

Render environment variable added, build command updated, redeployed

Knowledge base build script run once against the production database

Mobile app: API client function added

Mobile app: free-text input and reasoning trace panel added to the Recommendation screen

End-to-end test: mobile app → Render → Groq → tools → answer, confirmed working